

MEKDEL AMBA UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATICS

DEPARTMENT OF COMPUTER SCIENCE

INTERNSHIP REPORT ON

DESIGN AND DEVELOPMENT OF THE BDU SMART STORE MANAGEMENT SYSTEM

Item	Information
Student Name	Digis Birhan
Student ID	MAU/1602192
Company/Host Organization	Bahir Dar University (Wisdom)
Internship Duration	July 1 /11/2018 E.C– Augest 30/12/2018 E.C
Academic Advisor	Mr.Mebitu
Company Supervisor	Mr.Mekias
Date	July – August 2018 E.C

Gimba, Ethiopia

APPROVAL SHEET / DECLARATION

This section must be completed and signed by the intern, company supervisor, academic advisor or mentor, and department head. The guideline requires the student declaration to affirm originality and actual internship experience, together with the required approvals and company stamp. Do not sign this section until all information has been verified.

Part	Name	Signature	Date
Student/Intern	Digis Birhan		
Company Supervisor	Mr.Mekias		
AcademicAdvisor / Mentor	Mr.Mebitu		
Department Head	Mr.lawuyew		

Company Stamp: _____

ABSTRACT

This report presents an internship project focused on the analysis, design, development, integration, and testing of the BDU Smart Store Management System. The project is a web-based store and material-request management platform implemented with a React/Vite frontend and an Express/Node.js backend connected to MongoDB through Mongoose. The application separates responsibilities among six operational roles: administrator, requester, department head, property administrator, storekeeper, and clerk. Its central business process manages a material request from creation through verification, departmental approval, property authorization, assignment, item issue, transaction recording, and completion. The implementation includes authentication and role-based permissions, inventory and asset management, notifications, reports, document handling, audit logging, backup and restore facilities, system settings, and real-time communication support through Socket.IO. The source archive also contains automated server-side tests covering authentication sessions, backup metadata and authorization, error handling, security behavior, and upload/rate-limit controls. The internship provided practical exposure to full-stack development, database modeling, API design, access control, workflow validation, testing, debugging, and user-interface development. The project evidence reviewed for this draft supports the technical and functional description above; personal internship dates, supervisor information, company history, measured business impact, and authenticity evidence must be completed from the intern's own records before submission.

Keywords: BDU Smart Store, inventory management, material request workflow, role-based access control, MERN-style web application

TABLE OF CONTENTS

APPROVAL SHEET / DECLARATION II

ABSTRACT III

LIST OF FIGURES VI

LIST OF TABLES VII

CHAPTER 1: INTRODUCTION 1

1.1. Background 1

1.2. Purpose and Objectives of Internship 1

1.3. Duration and Location 2

1.4. Document Organization 2

CHAPTER 2: COMPANY BACKGROUND 3

2.1. Company History 3

2.2. Products / Services 3

2.3. Organizational Structure 4

CHAPTER 3: PROJECT SELECTION AND EXECUTION 8

3.1. Project Title and Description 8

3.2. Problem Statement 8

3.3. Objectives 9

3.3.1. General Objective 9

3.3.2. Specific Objectives 9

3.4. Methodology 9

3.5. Literature / Industry Review 10

CHAPTER 4: INTERNSHIP EXPERIENCE AND SPECIFIC WORK 11

4.1. Department and Role 11

4.2. Daily Responsibilities 11

4.3. Major Tasks / Projects Completed 11

4.4. Tools and Technologies Used 12
4.5. Challenges Faced 13
4.6. Solutions Implemented 13
4.7. Results and Achievements 14
5.1. Technical Skills Enhancement 17
5.2. Theoretical Knowledge Application 18
5.3. Problem-Solving Capability 18
5.4. Professional Skills Development 19
5.5. Teamwork and Collaboration 19
CHAPTER 6: CONCLUSION AND RECOMMENDATIONS 20
6.1. Overall Conclusion 20
6.2. Recommendations for the Company 20
6.3. Recommendations for the University 21
6.4. Personal Future Plans 21
6.5. Final Reflection 21
REFERENCES 22
APPENDICES 23
Appendix A: Fig:A.1 Lab assistance assist at work space 23
Appendix B: Project Source Inventory 23
Appendix C: Request Workflow Evidence 24
Appendix D: Current Web Interface Evidence 24
Appendix E: Required Original Materials Still to Be Added 26

LIST OF FIGURES page

Figure 2.1 Structure of BDU ---------- 5

Figure 2.2 Work flows of BDU ---------- 6

Figure 2.3 Collaborative Work Environment Evidence ---------- 7

Figure 4.1 Requester/User Dashboard ---------- 15

Figure 4.2 Department Head dashboard ---------- 15

Figure 4.3 Property Administrator Dashboard ---------- 16

Figure 4.4 Store Keeper Dashboard ---------- 16

Figure 4.5 Clerk Dashboard ---------- 17

Figure 4.6 System Administrator Dashboard ---------- 17

Figure A.1 Lab assistance assist at work space ---------- 23

Figure D.1 Store Related Visual Animation ---------- 25

Figure D.2 Campus/ Building Related Visual Animation ---------- 25

Figure D.3 Poly Campus Building Visual Animation ---------- 26

Figure D.4 Sport Field Visual Animation ---------- 26

LIST OF TABLES page

Table 2.1. Company History ---------- 3

Table 3.1. Methodology ---------- 10

Table 4.1. Tasks completed ---------- 12

Table 4.2. Tools ---------- 13

Table 4.3. Challenges Faced ---------- 13

Table 4.4. Achievements ---------- 14

Table 5.1. Skills ---------- 18

Table 5.2. Knowledge ---------- 18

Table B.1.Project Source Inventory ---------- 23

Table C.2. Work Flow Evidence---------- 24

CHAPTER 1: INTRODUCTION

1.1. Background

Internship is intended to connect academic learning with professional practice. In computing and informatics, this connection is especially important because software engineering concepts become meaningful when they are applied to requirements, interfaces, databases, APIs, security, testing, deployment, and user support. The internship project reviewed for this report is the BDU Smart Store Management System, a web application intended to coordinate material requests and store operations through a controlled digital workflow.

The reviewed source archive shows a substantial full-stack implementation rather than a single-page demonstration. The frontend is organized with React and Vite and includes reusable layouts, authentication context, language and dark-mode contexts, service modules, protected routes, role-specific dashboards, and multiple workspaces. The backend uses Express, Node.js, MongoDB/Mongoose, authentication middleware, role and permission middleware, controllers, services, scheduled low-stock alerts, file upload security, reporting, backup/restore, and Socket.IO integration.

The system addresses a business process in which a requester creates a material request, the request is verified and routed to the responsible department head, the approved request is authorized and assigned by the property administrator, items are prepared and issued by the storekeeper, and the clerk records and completes the resulting transaction. This workflow is represented explicitly in the backend documentation and is reinforced by request-state validation in the application.

1.2. Purpose and Objectives of Internship

The internship objectives should be stated in the student's own words. Based on the reviewed project, the following objectives are appropriate candidates for verification and refinement:

- To gain practical experience in full-stack web application development using React, Node.js, Express, and MongoDB.
- To apply database design and API-development concepts to a real material-request and inventory workflow.

- ❖ To understand and implement authentication, role-based authorization, permissions, audit logging, and secure file handling.
- ❖ To develop interactive role-specific dashboards and connect their interfaces to backend services.
- ❖ To improve software testing, debugging, documentation, and professional collaboration skills.

These objectives complement academic coursework by requiring the intern to move from isolated classroom exercises to an integrated application where changes in the data model affect APIs, workflow rules, user interfaces, security, and reporting. The final version of this section should also explain why the host organization was selected and what research the student performed before joining.

1.3. Duration and Location

Internship start date: July 1/11/2018 E.C. End date: Augest 30/12/2018 E.C. Total hours completed: 2:30 - 10:30. Workplace address: Peda ICT Lab class/ Main Campus. Work mode: HYBRID. These details are not established by the uploaded project archive and therefore must be supplied from official internship records.

1.4. Document Organization

Chapter 1 introduces the internship context and objectives. Chapter 2 documents the host organization and the work environment. Chapter 3 explains project selection, the problem addressed, objectives, methodology, and industry context. Chapter 4 presents the internship role, daily activities, major technical work, tools, challenges, solutions, and results. Chapter 5 reflects on technical, theoretical, problem-solving, professional, teamwork, leadership, and ethical benefits. Chapter 6 concludes the report and provides recommendations for the host organization, the university, and the intern's future development. References and appendices provide supporting evidence.

CHAPTER 2: COMPANY BACKGROUND

2.1. Company History

The uploaded guideline requires the history, establishment year, founders, milestones, mission, vision, and core values of the host organization. The BDU Smart Store source archive establishes the project identity and contains university-related visual assets, but it does not provide a verified corporate history. Therefore, this section must be completed from official host-organization documents or from the student's firsthand internship knowledge rather than inferred from the software repository.

Required information	**Student must verify**
Organization name	Bahir Dar University (BDU)
Establishment year	1963 E.C. (1971 G.C.)
Founders / establishing body	Established by the Government of Ethiopia through the merger and development of higher-education institutions in Bahir Dar.
Major milestones	1963 E.C.: Bahir Dar Polytechnic Institute was established. 1969 E.C.: Bahir Dar Teachers' College was established. 2000 E.C. (2008 G.C.): Bahir Dar University was established through the merger of Bahir Dar Polytechnic Institute and Bahir Dar Teachers' College.
Mission	To provide quality education, conduct problem-solving research, and provide community and professional services that contribute to national development.
Vision	To become a leading university recognized for excellence in education, research, innovation, and community engagement.
Core values	Excellence, integrity, professionalism, accountability, diversity, innovation, teamwork, and community service.

Table 2.1 Company History

2.2. Products / Services

From the software evidence, the operational context represented by the system includes material requests, item cataloging, inventory control, asset management,

request approvals, authorization, assignment, item issue, transaction recording, documents, notifications, reports, audit trails, system settings, and backup/restore. These functions should be described as the digital processes supported by the project. The host organization's broader products and services must be added from verified company information.

- Material/request management and approval tracking.
- Inventory and consumable stock management, including reorder-level information.
- Asset and issue management.
- Transaction and document recording.
- Reports, notifications, audit logs, system settings, and administrative controls.

2.3. Organizational Structure

Bahir Dar University is organized into several interconnected academic and administrative units. At the top of the university's governance structure is the Board of Governors, which provides institutional oversight. The President serves as the chief executive and provides overall leadership and coordination of university activities.

The university's major organizational areas include academic affairs, research and community service, administration and development, student services, and various executive and support offices. Academic activities are carried out through colleges, schools, faculties, institutes, and departments. Research and community-service activities are supported by research institutes, research centers, laboratories, and related programs.

Administrative and development activities are supported by directorates and offices responsible for areas such as human resources, finance, procurement, information and communication technology, planning, property management, and general services. In addition, the university provides student services, library services, and other institutional support services.

This organizational structure enables Bahir Dar University to coordinate its teaching, learning, research, community engagement, administration, and support services. The

different units work together under the university's leadership to achieve its educational and institutional objectives.



Fig:2.1 Structure of BDU

2.4. Workflows and Processes

Bahir Dar University (BDU) carries out its activities through a coordinated system of academic, research, administrative, student, and support services. The university's work is guided by its leadership and institutional policies and is implemented through colleges, institutes, departments, directorates, offices, and service units.

The major academic processes include teaching, learning, admission, registration, assessment, graduation, and other academic services provided through colleges, schools, faculties, and departments. Research and community-service activities involve conducting research, supporting research projects, disseminating knowledge, and providing services that contribute to the development of the surrounding community.

The university also performs important administrative processes, including human-resource management, financial management, procurement, property administration,

planning and development, information and communication technology services, and general institutional support. These processes help ensure that academic and research activities receive the necessary resources and services.

Student-related processes include admission and registration, academic support, student affairs, counseling, career development, and other services that support students throughout their university experience. Library and information services provide students, instructors, researchers, and staff with access to learning materials, digital resources, information services, and archival resources.

BDU's activities generally follow a continuous process of **planning, implementation, monitoring and evaluation, reporting, and improvement**. Through this process, the university identifies priorities, implements planned activities, evaluates their performance, prepares reports, and uses the results to improve future activities

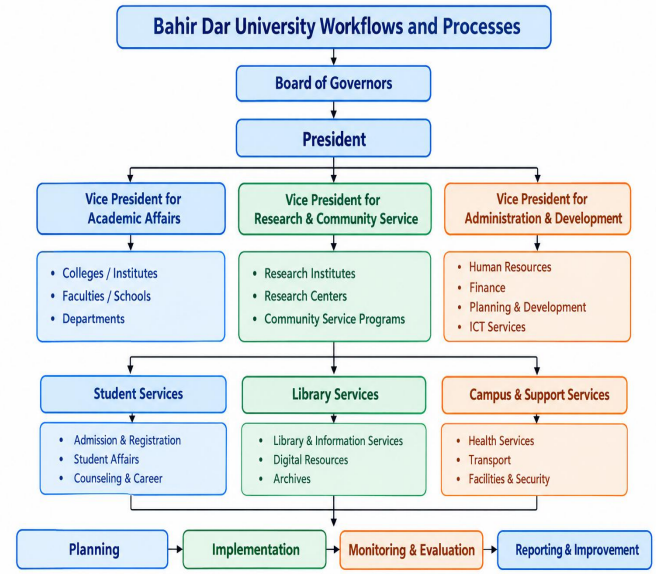


Fig:2.2 work flows of BDU

2.5. Company Culture

The repository cannot reliably establish the host organization's management style, social culture, communication norms, or office atmosphere. The intern should write this section from observation, supported where appropriate by photographs and meeting evidence. The guideline specifically requires a photograph of the company premises and the student's original organizational chart.

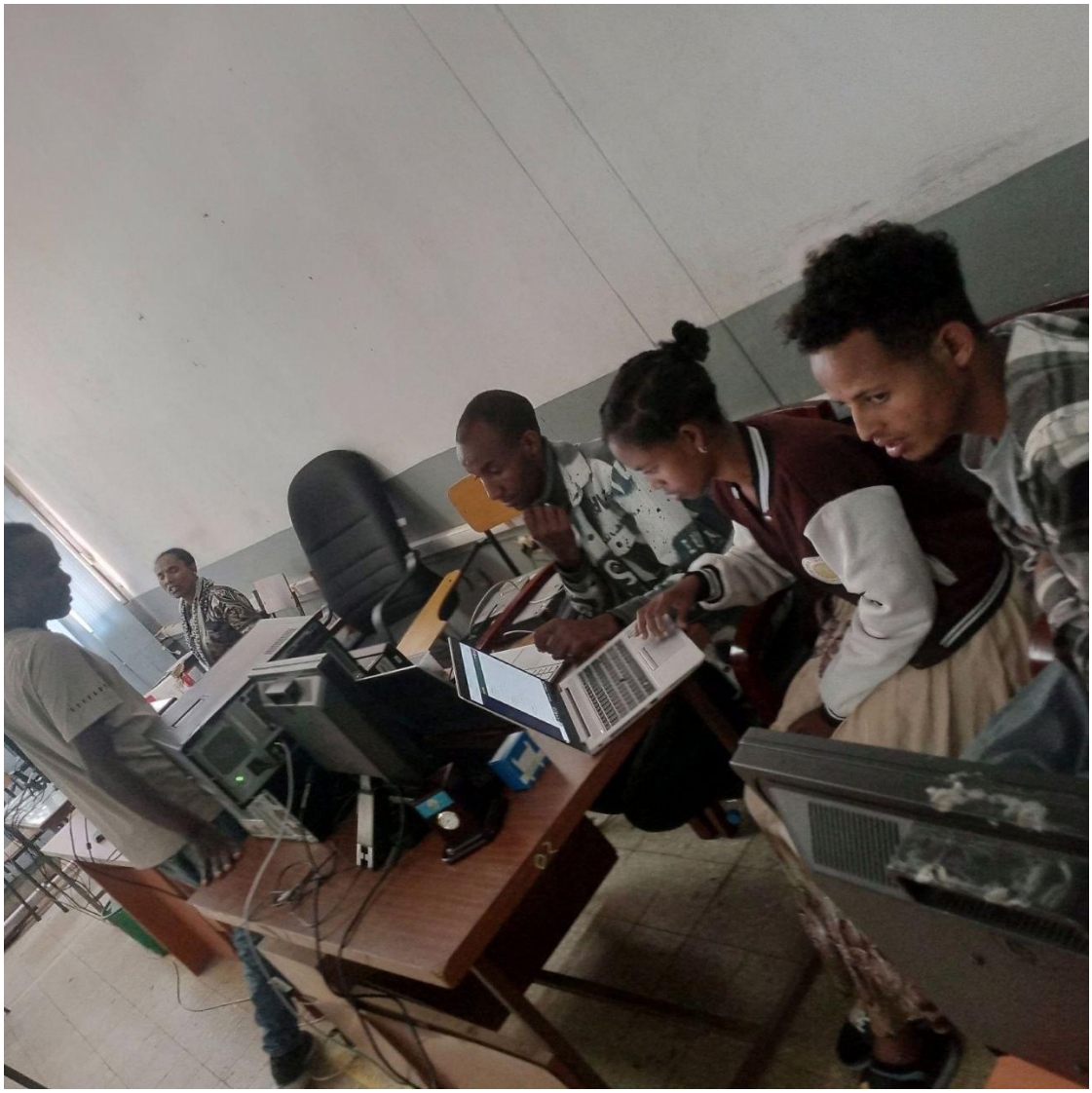


Figure 2.3. Collaborative work environment evidence supplied by the intern.

CHAPTER 3: PROJECT SELECTION AND EXECUTION

3.1. Project Title and Description

Project title: Design and Development of the BDU Smart Store Management System. The project is a web-based management platform for digitizing material requests, approvals, authorization, inventory operations, item issuance, transaction recording, documents, notifications, reporting, and administrative controls. The frontend contains public pages such as Home/Landing, About, Features, How It Works, Contact, Login, Forgot Password, and Reset Password, followed by protected role-specific workspaces.

The project is structured as a client-server application. The React frontend communicates with an Express API. The server exposes routes for authentication, dashboards, reports, users, requests, inventory, transactions, documents, assets, notifications, settings, backup, master data, support, AI-related functionality, and smart operations. MongoDB/Mongoose models represent the persistent domain objects, while middleware provides authentication, authorization, error handling, rate limiting, and upload security.

3.2. Problem Statement

The project addresses the operational difficulty of managing material requests and store activities when request initiation, verification, approval, authorization, assignment, issue, transaction recording, and supporting documents are not controlled through one integrated workflow. A fragmented process can make it difficult to know who is responsible for a request, what stage it has reached, which quantities were approved or issued, what stock remains, and which actions have been recorded for audit purposes. The BDU Smart Store system was therefore designed to provide a structured, role-aware and traceable digital process.

3.3. Objectives

3.3.1. General Objective

To design and develop an integrated web-based store management system that digitizes material requests, approvals, inventory operations, item issuance, transactions, and administrative monitoring.

3.3.2. Specific Objectives

- ❖ Implement secure authentication and role-based access control for the six defined application roles.
- ❖ Digitize the material-request lifecycle from request creation through verification, approval, authorization, assignment, issue, transaction recording, and completion.
- ❖ Provide inventory, item, category, location, asset, document, notification, and reporting capabilities appropriate to store operations.
- ❖ Implement audit logging, backup/restore controls, upload validation, rate limiting, and centralized error handling.
- ❖ Provide interactive role-specific dashboards and reusable frontend services for communication with backend APIs.

3.4. Methodology

The implementation can be described as an iterative software-development process consisting of requirements understanding, workflow modeling, data modeling, backend/API development, frontend development, integration, security hardening, testing, and refinement. The source structure supports this interpretation through dedicated models, routes, controllers, services, middleware, frontend pages/components, and automated tests.

1. Requirements and workflow analysis: identify actors, responsibilities, request states, permissions, and business rules.
2. Data modeling: define Mongoose schemas for users, requests, request items, inventory, assets, transactions, documents, notifications, audit logs, settings, locations, categories, roles, and related entities.
3. Backend implementation: create Express routes, controllers, middleware, services, error handling, authentication, authorization, and workflow validation.

4. Frontend implementation: build public pages, authentication pages, protected routes, role-specific dashboards, navigation, tables, forms, reports, and service modules.
5. Integration: connect frontend services to REST APIs and add Socket.IO-based real-time capabilities.
6. Security and reliability: test authentication, permission boundaries, rate limiting, upload validation, error responses, and backup metadata behavior.
7. Verification and refinement: execute automated tests, inspect errors, correct workflow and UI behavior, and document the resulting system.

Phase	**Main activities**	**Evidence in archive**
Planning	Actors,workflow, permissions, modules	REQUEST_WORKFLOW.md, role configuration
Design	Data entities and application structure	19 Mongoose models, frontend component/page structure
Development	REST APIs, UI, dashboards, services	16 route groups, 15 controllers, 127 frontend JS/JSX source files
Security	Auth, RBAC, rate limiting, upload checks, errors	Middleware and security tests
Testing	Automated authentication, security, backup, error tests	5 test files and test outputs

Table 3.1 Methodology

3.5. Literature / Industry Review

For final submission, this section must be expanded with at least ten credible sources in APA 7th edition as required by the university guideline. The current uploaded project archive does not contain ten academic or industry references, so no unsupported literature claims are inserted here. The final version should review modern inventory-management practices, role-based access control, web application security, RESTful API design, database-backed workflow systems, audit logging, backup/recovery, and usability of enterprise dashboards. The student should also attach project-planning evidence such as Trello, Jira, MS Project, meeting notes, or equivalent original records.

CHAPTER 4: INTERNSHIP EXPERIENCE AND SPECIFIC WORK

4.1. Department and Role

Department: [ENTER VERIFIED DEPARTMENT]. Internship title: [ENTER VERIFIED TITLE]. The technical work represented in the project is consistent with a software-development or information-technology placement involving full-stack application development, database-backed services, user interfaces, security, and testing. The student's official placement information and the rationale provided by the host organization must be inserted from internship records.

4.2. Daily Responsibilities

A final report should describe the student's actual daily routine rather than assume a fixed schedule. The following activity categories are supported by the project evidence and may be used to organize the student's own time log:

- Reviewing requirements and clarifying workflow rules.
- Designing or updating React interfaces and dashboard navigation.
- Developing or modifying Express controllers, routes, services, and middleware.
- Creating or refining MongoDB/Mongoose models and validation rules.
- Integrating frontend service calls with backend APIs.
- Testing authentication, permissions, request transitions, inventory behavior, documents, and security controls.
- Debugging frontend/backend integration issues and documenting solutions.
- Participating in review, feedback, demonstrations, or handover activities.

4.3. Major Tasks / Projects Completed

The uploaded archive provides evidence of the following major technical work areas. The student should add exact dates and distinguish work personally completed from work performed by other team members.

Major task	Technical evidence	Contribution to system
Authentication and session handling	JWT/auth routes, AuthContext, auth tests	Secure login, session restoration, logout, protected access
Role-based access control	roles.js, role middleware, ProtectedRoute	Different capabilities for six roles
Request workflow	workflow utilities, MaterialRequest model, request controller	Controlled multi-stage material request lifecycle
Inventory management	Inventory model, inventory routes/services, Storekeeper dashboard	Stock visibility, updates, issuing, low-stock behavior
Transaction management	StockTransaction model, transaction controller, Clerk transactions view	Record and review stock movements
Document management	Document routes/controller, PDF issue-slip files	Store and retrieve supporting documents
Audit and administration	AuditLog model, admin views	Trace system actions and manage users/settings
Backup and restore	BackupMetadata, backup/restore services, tests	Controlled backup lifecycle and restore evidence
Notifications / real time	Notification model, Socket.IO, notification services	Status and operational notifications
Security hardening	Rate limiter, upload validation, error middleware, tests	Reduce abuse and unsafe input/file behavior

Table 4.1 Tasks completed

The archive contains a broad frontend structure, including dedicated components for administrator management, department-head workspaces, property-administrator operations, storekeeper inventory operations, clerk transactions/documents, and requester request management. This supports describing the project as a multi-role management system rather than a single dashboard.

4.4. Tools and Technologies Used

Category	Technology / tool	Observed purpose
Frontend	React	Component-based user interfaces and dashboards
Frontend tooling	Vite	Development/build tooling
UI styling	Tailwind CSS	Responsive interface styling
Icons/interaction	Lucide React, Framer Motion	Icons and interface animation
Routing	React Router	Public/protected application navigation
Backend	Node.js, Express	HTTP API and server-side application logic
Database	MongoDB, Mongoose	Persistent domain data and schema modeling

Authentication	JWT, bcryptjs	Identity verification and password hashing
Realtime	Socket.IO	Realtime server/client communication
Files	Multer, PDFKit, fs-extra	Uploads, PDF generation, and file operations
Email	Nodemailer	Email-related system capability
Testing	Node test runner, assertions	Automated backend/security verification
Versioning / project evidence	Git-related project structure	Development history where repository history is available

Table 4.2 Tools

4.5. Challenges Faced

The archive demonstrates that the project involved non-trivial engineering challenges. The following are technical challenges evidenced by the implementation; the student should revise them to match the actual problems personally encountered and add dates, symptoms, and impact.

Challenge	**Observed technical manifestation**	**Likely impact**
Complex multi-role workflow	Many states and role-specific transitions	Risk of invalid transitions or unauthorized actions
Authentication and session consistency	JWT token, cookie/session restoration, protected routes	Login/session failures can block all protected features
Security boundaries	Permissions, department ownership, admin-only operations	Incorrect authorization could expose sensitive operations
Frontend/backend integration	Many service modules and API endpoints	Network/configuration mismatches can cause dashboard or login failures
File and document security	Upload validation and document handling	Unsafe uploads or incorrect content types can threaten reliability/security
Inventory consistency	Approved, issued, and remaining quantities must agree	Incorrect stock movement can create unreliable inventory records

Table 4.3 Challenges Faced

4.6. Solutions Implemented

- Centralized workflow validation was used to restrict allowed request-state transitions by role.
- Protected routes and backend authorization were combined so frontend visibility was not the only security boundary.

- ❖ The request model stores requester, department, responsible head, assigned storekeeper, issue information, approval trail, and logs to support traceability.
- ❖ Central error middleware standardizes API error responses and hides unexpected database details.
- ❖ Authentication rate limiting was implemented to block repeated authentication attempts.
- ❖ Uploaded files are checked against their declared content type and rejected when the content does not match.
- ❖ Backup metadata records verification status, restore attempts, creator identity, retention information, and restoration state.
- ❖ Inventory records enforce a relationship to an existing consumable item and maintain reorder-related quantities.

The project test suite provides additional evidence that these solutions were not purely visual. Authentication/session restoration, backup metadata, error handling, security boundaries, rate limiting, and upload validation are explicitly exercised in automated tests.

4.7. Results and Achievements

The archive demonstrates substantial implementation coverage. It contains 127 frontend JavaScript/JSX source files, 70 backend source files under the main server source tree, 19 backend models, 16 route groups, 15 controllers, 5 backend services, and 5 automated test files. It also contains generated issue-slip PDFs and a built frontend distribution.

Achievement	**Evidence / status**
Multi-role architecture	Six application roles defined with role-specific permissions
Request workflow	Canonical state transitions documented and validated
Inventory capability	Inventory model and storekeeper views/services implemented
Transaction capability	Transaction model/controller and clerk transaction UI implemented
Administrative control	User, roles/permissions, audit, settings, backup/restore views exist
Security testing	Dedicated authentication and security test files present
Document generation/storage	Document service/controller and issue-slip PDFs present
Responsive/public web structure	Landing, About, Features, How It Works, Contact, and authentication pages present

Table 4.4 Achievements

Quantified business outcomes such as percentage reduction in processing time, inventory accuracy, user adoption, or paper reduction are not established by the source archive. They must be measured from actual internship/project evaluation data before being reported as results.

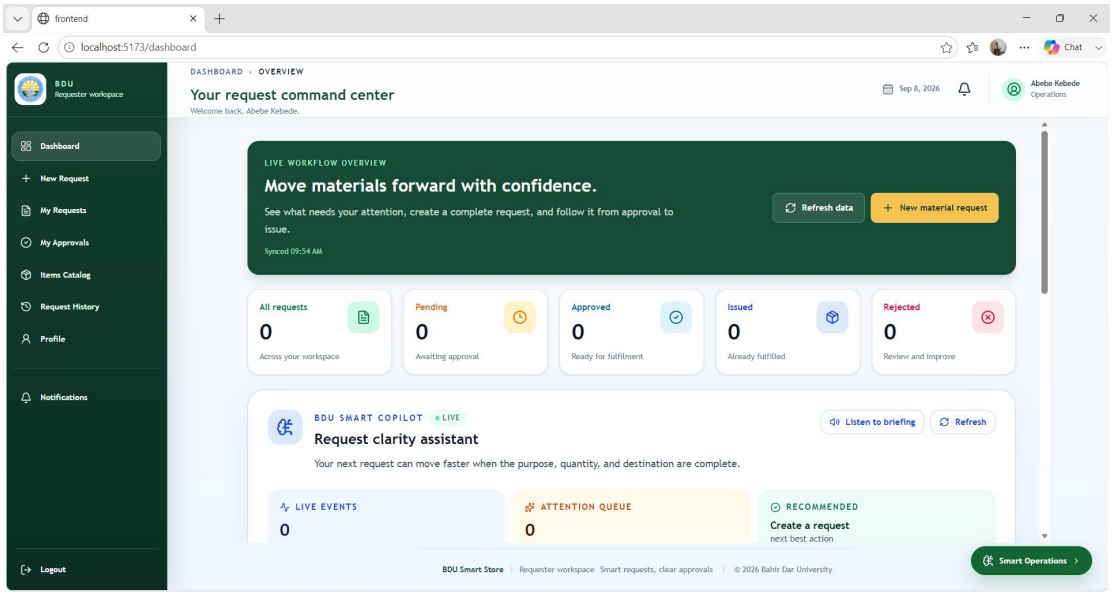


Figure 4.1. Requester dashboard and role-specific interface evidence from the running system.

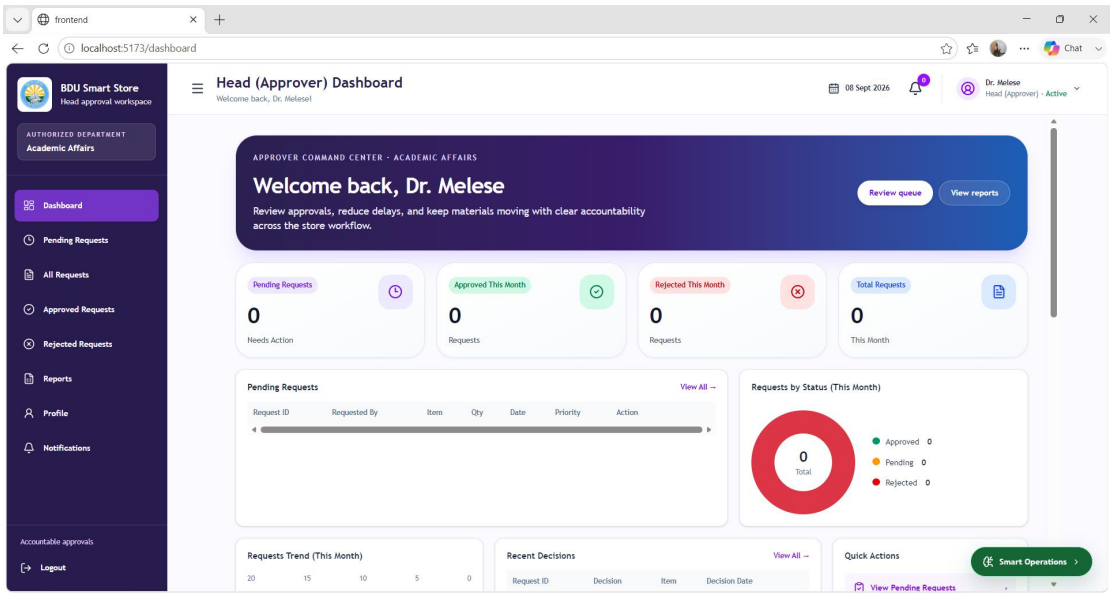


Figure 4.2. Department Head dashboard

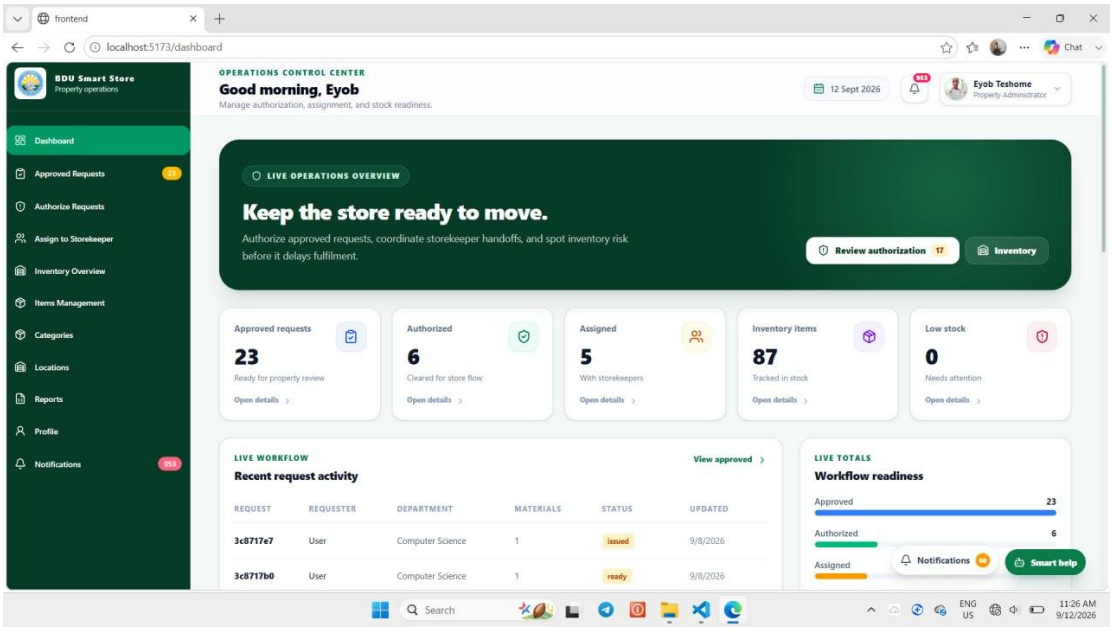


Figure 4.3. Property Administrator dashboard

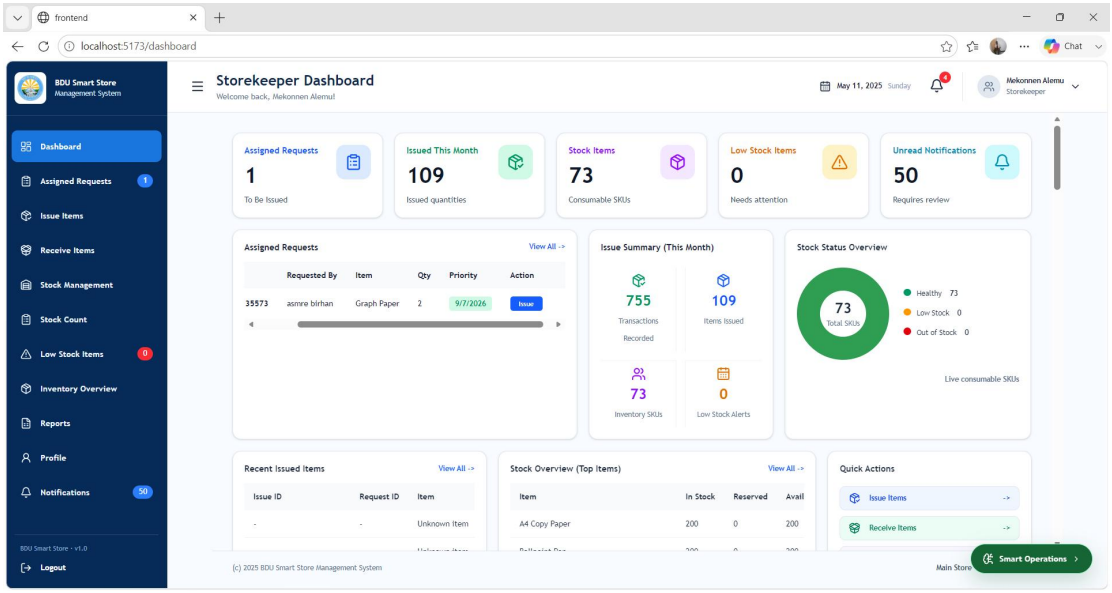


Figure 4.4.Storekeeper dashboard

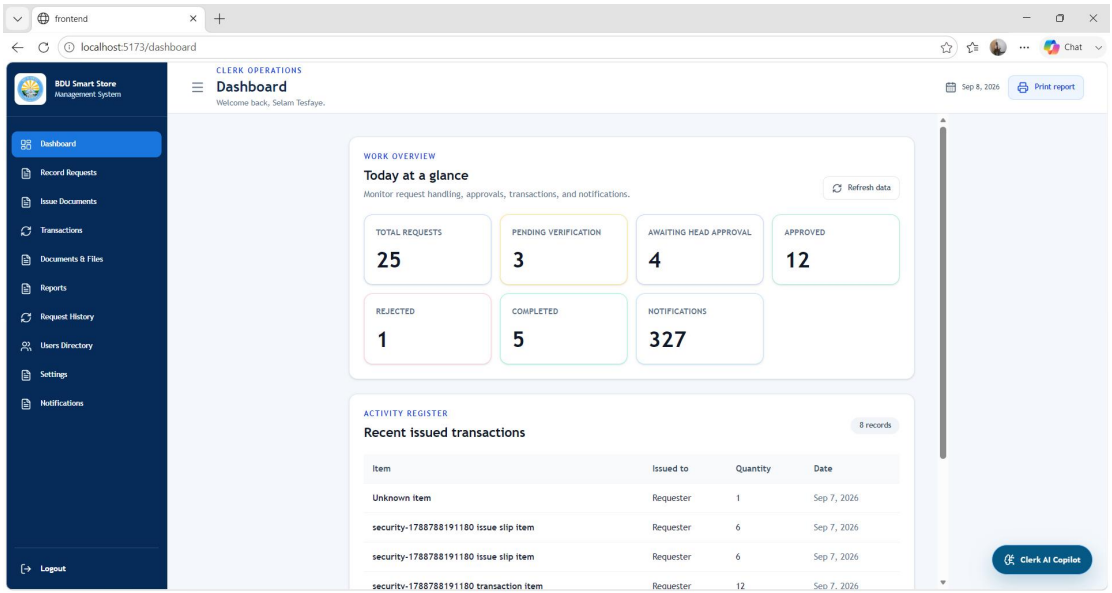


Figure 4.5. Clerk dashboard

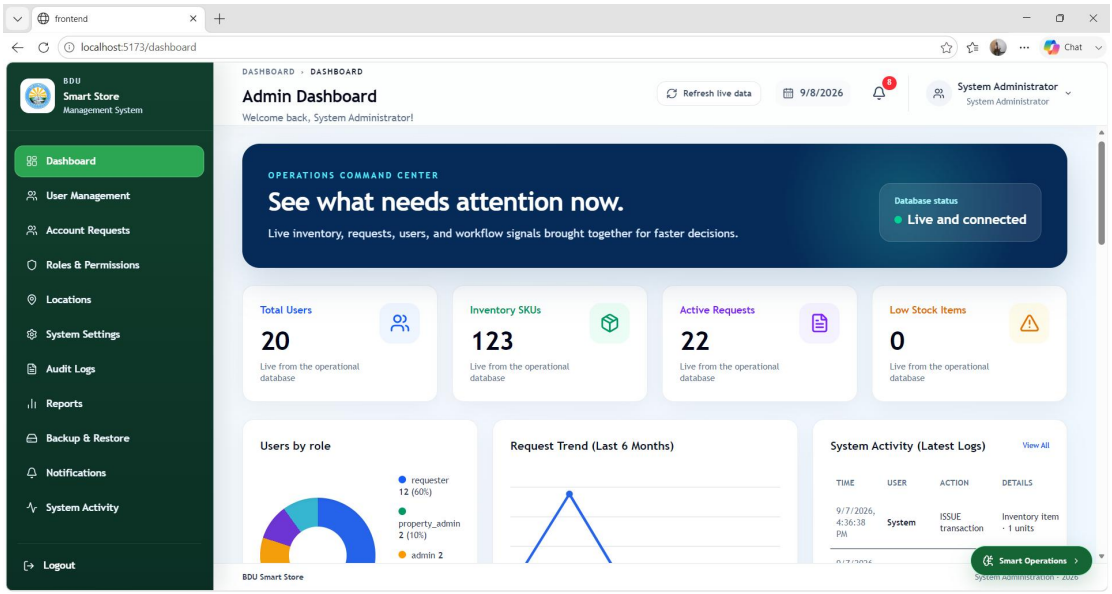


Figure 4.6. Administrator dashboard

CHAPTER 5: BENEFITS GAINED

5.1. Technical Skills Enhancement

The project provides practical exposure to several technical areas. The exact before-and-after competency levels should be assessed honestly by the intern. A useful structure is shown below.

Skill	Before internship	After internship	Evidence
React / component development	basic	improved	Frontend source/components
REST API development	basic		Express routes/controllers
MongoDB/Mongoose	basic	improved	19 Mongoose models
Authentication/RBAC	basic	improved	Auth and role configuration
Testing/debugging	basic	improved	Automated test suite
Security practices	basic	improved	Rate limiting/upload validation/error handling

Table 5.1 Skills

5.2. Theoretical Knowledge Application

Academic concept	Practical application in project
Database modeling	Mongoose schemas for requests, inventory, users, assets, transactions, documents, and logs
Software engineering	Modular frontend/backend structure and separation of concerns
Web development	React UI connected to Express REST APIs
Information security	JWT authentication, RBAC, permissions, rate limiting, upload validation
Software testing	Automated tests for authentication, security, errors, and backup behavior
Human-computer interaction	Role-specific dashboards, navigation, tables, status indicators, and forms
Systems analysis	Explicit request lifecycle and actor responsibilities

Table 5.2 knowledge

One important practical lesson is that classroom concepts interact in a real system. A change to a request status is not only a database change; it affects authorization, notifications, dashboard labels, audit information, and the next permitted action. Similarly, inventory quantities must remain consistent with approved and issued quantities.

5.3. Problem-Solving Capability

The project strengthened problem-solving through workflow validation, error normalization, security controls, and automated testing. Rather than allowing each user interface to decide independently what is permitted, the backend centralizes important business rules. This reduces the risk that a client-side change can bypass server-side controls.

A second problem-solving lesson concerns failure diagnosis. A full-stack application can fail at several layers: browser routing, frontend service configuration, HTTP/API communication, authentication, server routing, database connectivity, or data validation. Effective troubleshooting therefore requires isolating the failing layer before changing code.

5.4. Professional Skills Development

The internship environment should be used to demonstrate improvements in written communication, verbal communication, presentation, time management, documentation, and task organization. The student should insert specific examples such as explaining a workflow to a supervisor, presenting a completed dashboard, writing technical notes, or meeting a sprint deadline.

5.5. Teamwork and Collaboration

The final report should identify actual team interactions and the student's contribution. Examples may include requirements discussions, code review, UI feedback, database/API coordination, testing sessions, or project demonstrations. Any claims about conflict resolution or different working styles must be based on real experiences.

5.6. Leadership Experience

Leadership during an internship does not necessarily mean managing people. It may include taking ownership of a module, organizing a debugging session, proposing a workflow improvement, preparing a demonstration, or helping another team member understand a technical issue. Insert one or more verified examples from the internship.

5.7. Work Ethics and Professionalism

The project highlights professional responsibilities around access control, auditability, data integrity, file security, and backup/recovery. The student should connect these technical concerns to the organization's actual code of conduct, confidentiality requirements, data-handling rules, and workplace standards. Certificates or formal recognition should be attached only when genuinely received.

CHAPTER 6: CONCLUSION AND RECOMMENDATIONS

6.1. Overall Conclusion

The BDU Smart Store Management System represents an integrated attempt to digitize a multi-stage store and material-request process. The reviewed implementation combines a React/Vite user interface with an Express/Node.js backend and MongoDB/Mongoose persistence. It defines six application roles and implements a controlled request lifecycle covering creation, verification, approval, authorization, assignment, item issue, transaction recording, and completion. Supporting capabilities include inventory, assets, documents, notifications, reporting, audit logs, settings, backup/restore, and security middleware.

The internship value of this project lies in the integration of multiple areas of computing practice. The work requires understanding both the user-facing experience and the server-side business rules, while maintaining consistency between database state, permissions, workflow, and interface behavior. The final assessment of whether the internship objectives were fully achieved should be written by the student using supervisor feedback, project milestones, and personal reflection.

6.2. Recommendations for the Company

8. Continue formalizing business workflows and keep workflow documentation synchronized with deployed application behavior.
9. Maintain automated regression tests whenever request states, permissions, inventory rules, or authentication behavior changes.
10. Use real operational metrics to evaluate the system, including request-processing time, stock accuracy, issue turnaround time, and user adoption.
11. Establish a documented production backup, verification, retention, and restore schedule and periodically test restoration.
12. Continue strengthening security through least-privilege permissions, secure secret management, audit review, and controlled file uploads.
13. Standardize dashboard design and navigation across roles while keeping role-specific actions clearly separated.
14. Provide structured user training and operational documentation before broad deployment.

These recommendations should be supported in the final submission with photographs of the intern performing tasks, screenshots of actual work outputs, signed task completion forms, and relevant communication evidence, as required by the university guideline.

6.3. Recommendations for the University

15. Provide earlier project-planning guidance so students enter internships with clear requirements, milestones, and evidence-collection plans.
16. Strengthen practical coverage of full-stack integration, authentication, role-based authorization, testing, deployment, and security.
17. Encourage students to maintain weekly technical journals that record tasks, challenges, solutions, and supervisor feedback.
18. Provide a standard evidence checklist early in the internship so students collect photographs, attendance records, meeting evidence, and deliverables throughout the placement.

6.4. Personal Future Plans

Based on the technical scope of the project, future development areas may include stronger TypeScript practice, automated frontend testing, CI/CD, containerization, cloud deployment, advanced security, observability, performance optimization, and scalable database design. The final version should identify the student's own career direction and the specific skills to be developed next.

6.5. Final Reflection

The most valuable lesson from an integrated software project is that successful systems are built around real processes, not isolated screens. A dashboard is useful only when the underlying workflow, permissions, data, and operational responsibilities are consistent. The internship should therefore be reflected upon as an opportunity to develop both technical competence and professional responsibility.

REFERENCES

The university guideline requires a minimum of ten credible sources and APA 7th edition formatting. The uploaded project archive does not contain a verified bibliography of ten external sources. The following entries are therefore placeholders to be replaced with the actual academic papers, books, standards, official technology documentation, industry reports, and company documents consulted by the student.

[REFERENCE 1 — APA 7th edition source on software engineering / systems development]

[REFERENCE 2 — APA 7th edition source on inventory or supply-chain information systems]

[REFERENCE 3 — APA 7th edition source on role-based access control]

[REFERENCE 4 — APA 7th edition source on web application security]

[REFERENCE 5 — APA 7th edition source on RESTful APIs / web services]

[REFERENCE 6 — APA 7th edition source on database management]

[REFERENCE 7 — APA 7th edition source on software testing]

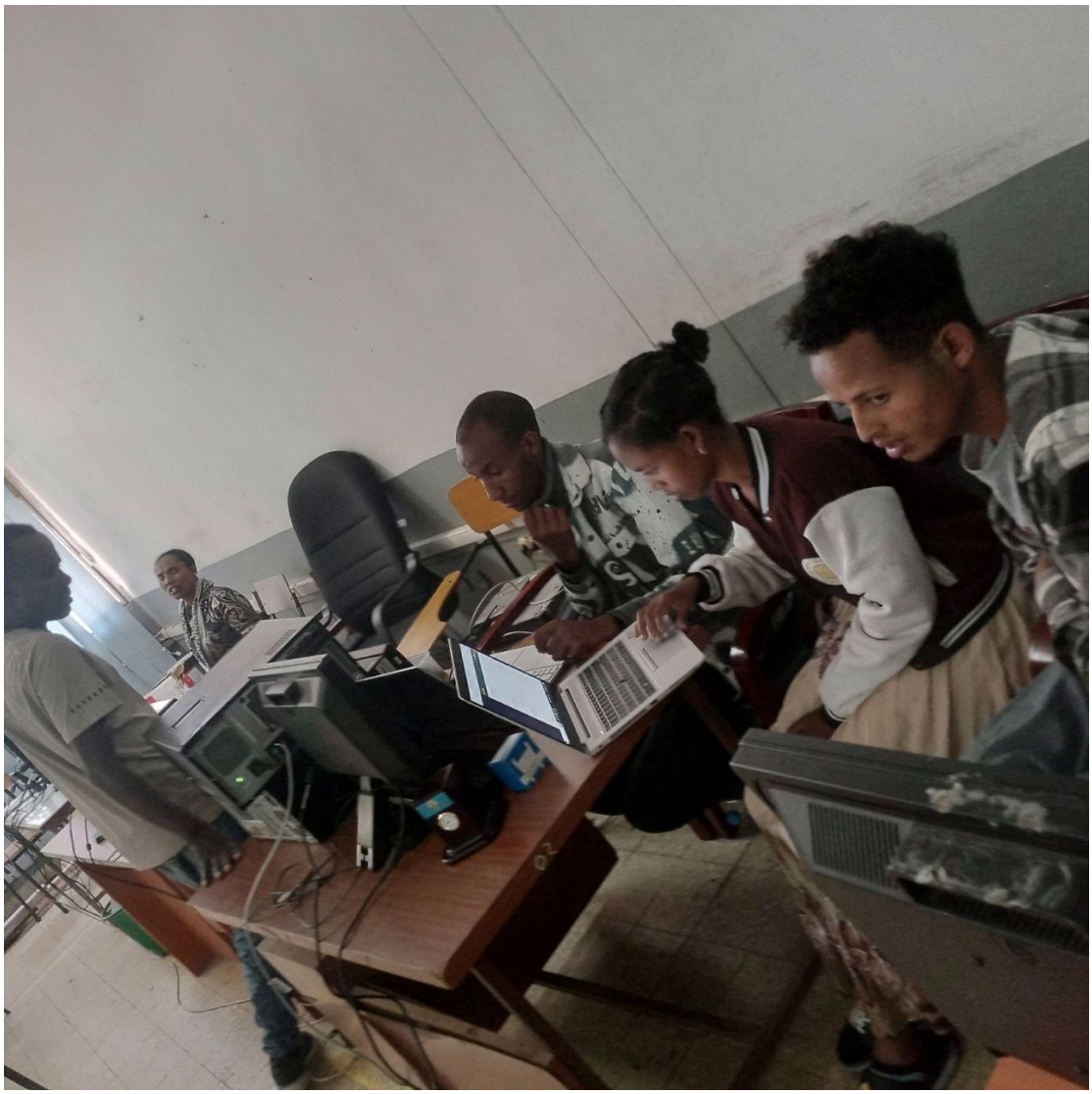
[REFERENCE 8 — APA 7th edition source on usability / dashboard design]

[REFERENCE 9 — APA 7th edition source on backup and disaster recovery]

[REFERENCE 10 — APA 7th edition source on enterprise information systems]

APPENDICES

Appendix A: Fig:A.1 Lab assistance assist at work space



Appendix B: Project Source Inventory

Area	Count observed in archive
Frontend JS/JSX source files	127
Backend source JavaScript files under server/src	70
Backend Mongoose models	19
Backend route groups	16
Backend controllers	15
Backend service files	5
Automated test files	5

Table B.1 Project Source Inventory

These counts are a structural inventory of the uploaded archive and are included to make the technical description auditable. They are not measures of quality, productivity, or individual contribution.

Appendix C: Request Workflow Evidence

Stage	Role	Allowed next action
Draft	Requester	Submit material request
Pending clerk verification	Clerk	Verify request information
Pending head approval	Responsible Department Head	Approve, reject, or request correction
Approved head	Head / Property Administrator	Move toward authorization
Authorized	Property Administrator	Authorize and assign
Assigned	Property Administrator / Storekeeper	Assign request to storekeeper
Ready	Storekeeper	Prepare item for issue
Partially issued / Issued	Storekeeper	Issue approved quantities
Transaction recorded	Clerk	Record the transaction
Completed	Clerk	Complete the request record

Table C.1 Workflow Evidence

Source evidence for the workflow is the project's REQUEST_WORKFLOW.md and the corresponding backend workflow/transition implementation. The final report should add the student's original workflow diagram if the university requires an original diagram.

Appendix D: Current Web Interface Evidence

The uploaded archive contains frontend public pages and role-specific dashboard components. The images below are assets included in the project distribution; they should not be presented as authenticity evidence for the internship itself.



Appendix Figure D.1. Store-related visual asset included in the frontend distribution.



Appendix Figure D.2. Campus/building visual asset included in the frontend distribution.



Appendix Figure D.3. Building visual asset included in the frontend distribution.



Appendix Figure D.4. Campus aerial visual asset included in the frontend distribution.

Appendix E: Required Original Materials Still to Be Added

- Original organizational chart showing the host organization's departments and the intern's placement.
- Original system architecture diagram, ER diagram, activity/workflow diagram, or other diagrams required by the department.
- Screenshots captured from the running website for each relevant role dashboard and key workflow screen.

- Supervisor correspondence, meeting notes, planning evidence, and signed task records.
- Exact internship dates, location, total hours, official position, supervisor, advisor, and department.
- At least ten verified APA 7th edition references.
- Measured project outcomes and supervisor feedback.

IMPORTANT: The university guideline explicitly states that generative AI tools must not be used to draft or generate the submitted internship report or its images/diagrams. This document is therefore provided as a structured technical draft based on the uploaded project archive and guideline. Before academic submission, the student must rewrite the narrative in their own words, verify every factual statement against personal records, insert original evidence, obtain required signatures/stamps, and comply with the university's academic-integrity requirements.